Directing a multi-machine computation with Claude Code: how stuck work moved in a computation of Choptuik's echo period

Nathan Moinvaziri[1]

[1] *Independent researcher*

(Dated: November 2026; unreviewed draft)

I directed a numerical research computation, the echo period $\Delta(D)$ of Choptuik's critical solution in D spacetime dimensions, from $D = 3.04375$ up past $D = 200$, with Claude, Anthropic's model, running in Claude Code on several machines. A coordinating Claude session wrote the code for the tasks I set, runner sessions on the other machines ran them and reported through an append-only file, and subagents took the reviews, searches and builds I asked for. The work stuck at 31 points. For each I give what stalled, what moved it and what that changed in the way of working. Twenty were moved first by a prompt of mine, eight by Claude's own diagnosis and three by another session or an outside protocol. I sort the 145 prompts of mine that changed the course of the work into ten kinds, from persistence and reframing to asking why, raising the scale, cutting delay, adding resources, bringing in my own tools, setting rules, correcting Claude and asking to understand, and I state the limits of the approach.

I. INTRODUCTION

When the collapse of a massless scalar field is tuned to the threshold of black-hole formation, the evolution approaches a universal solution that repeats itself on ever smaller scales, each echo a copy of the last shrunk by e^{Δ} [1]. The repository `choptuik-large-d` [2] computes Δ as a function of the number D of spacetime dimensions, and its physics is in two companion drafts [3, 4]. Each point is a periodic boundary-value problem solved by Newton's method from a prediction made at the last point. At large D the mass mode amplifies errors at the horizon by a gain $G \approx 0.7^{-(D-3)}$, so the arithmetic must carry more digits as D grows, from double-double [5] (about 32 digits) through quad-double [6] (about 64) to octo-double (about 126). At low D one point costs 4 to 10 hours.

The project began in a chat on claude.ai (Sec. II). Over the following three weeks I directed Claude sessions running in Claude Code [7] on five machines, and Claude wrote the code to my direction. The lowest converged point is $D = 3.04375$, and the large-D chain passed $D = 200$ and was still climbing at the time of writing. The work stuck at 31 points (Sec. V and Appendix A).

The sources are the chat where the project began [8], the coordination log `comms.txt`, the git history, the notes in `notes/`, the raw-data repository [9] and the transcripts of the Claude Code sessions on the coordinator machine. The other machines' side is known from their entries in the log. Every number, quotation and date is listed with its source in `paper/ai/SOURCES.md`.

II. HOW IT BEGAN

I began the chat with general questions, such as "*Are you able to make new discoveries about physics?*", and then asked "*Could it be done if I asked you to do it*". Claude offered three projects in general relativity that one machine could handle, a black-hole renderer, orbits in curved spacetime, and toy numerical relativity, the collapse of a scalar field that reproduces Choptuik's result. I picked the last, "*Do a toy numerical reality.*" From 177 runs the simulation reproduced Choptuik's mass scaling, with exponents of 0.377 and 0.378 for two pulse shapes against the accepted 0.374, and an echo period $\Delta \approx 3.30$ against 3.445, limited by a fixed grid that resolved about one echo. I remarked "*It is interesting that you focused on black holes when I was willing to accept any visual simulation of reality*" and steered to two side projects, a solar system and a gravity sandbox.

Then I asked "*Any new math we can draw from this?*" Claude answered that everything so far had reproduced known results and listed three directions, recommending the first, a proof that a known three-body orbit exists. I chose another, "*Explain the collapse numbers*". Claude proposed a plan that began with a literature check, and I let it run. The check changed the target, since a 2026 paper had already computed Δ for continuous D between 3.05 and 5.5 (see [3]). The next step of the plan computed the critical solution directly, as a periodic problem rather than a simulation. At 63 Fourier modes it gave $\Delta = 3.4454523972$ in four dimensions, nine digits from the best published value, 3.445452402, and the scan in D began.

I set the target, a formula, and kept pushing toward it, with "*Yes continue to find the formula you are doing great don't give up. It exists*", "*Could it be a sum?*", "*What if is a combination of a small sum and large sum?*" and "*Where is the formula?*" Claude turned each suggested form into a fit tested on dimensions it had not seen, as when

a fit to $D \le 8$ predicted the computed values out to $D = 10.1$ to within 2×10^{-5}, a habit that later became the held-out and planted-truth tests. When the scan stalled near $D = 9.3$, Claude said it would look into it later, and I wrote "*Continue searching*". The diagnosis that followed turned the stall into a finding. The condition that selects Δ weakens exponentially with D, since the competing solutions behave like $(1-x)^s$ near the horizon with s growing like $0.6\,(D-2)$. So no plain series in $1/D$ can fix Δ, an idea that became Proposition 1 of the later work, and double precision could fix Δ only to about 10^{-4} near $D = 10$. The fitted formulas themselves did not last. The first one for Δ fell like $(D-2)^{-1/4}$, while the later data give a finite limit, $\Delta_\infty = 1.643893746$ [3].

A formula needed many precise points, and I pushed on speed, with "*can we speed up the runtime by using SIMD?*", "*Does it make sense to rewrite in Rust?*" and "*With a subagent make a purpose build Rust build and measure speed.*", the first subagent of the project. Then I decided to leave the chat. I put what it had produced into a repository of my own, whose first commit is "Import the DSS residual kernel", asked "*How can I switch you to my local machine?*" and linked the chat to my laptop, "*You are now local on my computer. It has AVX512 too which is nice. Continue our work.*" When the scans used a fifth of the processor, I asked for more processes and threads. Within hours I had a cloud session and the laptop exchanging entries in `comms.txt`, and the setup of the next section grew from that pair.

III. THE SETUP

Roles.

- *Me.* I set the goals and the standards of evidence and chose among the options the coordinator put to me, by prompts and by answering its multiple-choice questions. I chose the machines and when to add them, merged and released the arithmetic library `multidouble` [10], and woke, plugged in and set up machines.
- *The coordinator*, one Claude Code session at a time, first on a cloud machine and then on the Mac, wrote the code I asked for in its own worktrees, merged results, kept the notes, queued numbered tasks, T1 to T135, and reported to me every hour.
- *The runners*, one session per other machine, ran their tasks, posted results and pushed raw output. They did not edit code, by my rule, "*make a note that the other machines are running a lower model than you are, so it is best for you to write any code and not them.*"
- *Subagents* were sessions the coordinator started at my request for one bounded task, each returning a single report.

TABLE I. The machines, by their names in `comms.txt`, with the entries each posted (headers counted at commit 07eff89). The laptop posted as `local` at first.

name	machine	role	entries
`cloud`	cloud container, 2 cores, 6.27 GB	coordinator at first; afterwards the coordinator's name on the Mac	293
`win`	Windows laptop, 4 cores and 8 threads, 31.8 GB	runner from the start: low D, builds, second paths	318
`mac`	Mac, Apple M5, 10 cores	runner, then coordinator	221
`win2`	Windows desktop, 20 cores, 95.6 GB	runner in the last week: the large-D chain, low-D legs	134
`macm3`	Mac laptop, Apple M3, 8 cores, 24 GB	runner on the last day: a second path past $D = 193.77$	8

The repositories as the record. Everything the work depends on lives in two git repositories (Fig. 1).

- *The main repository* holds the code, the notes, the papers, the derived tables such as `hp/data/delta_all.csv`, and `comms.txt`, the only channel between sessions. Every message is a commit, so the log is ordered, timestamped, append-only, addressed by its header, and survives any restart of a session or a machine. A writer pulls, appends, commits only that file and pushes.
- *The raw-data repository* holds every run's output, one folder per task and machine with a README, indexed by `INDEX.md`. No result may live only on a machine's disk, and `raw_check.py` and the writers' path check enforce it, because the records of 757 early points that never left the cloud machine's disk were lost.
- *Worktrees and branches.* Each session and subagent writes in its own worktree and backs up to a `wip/` branch, and only the coordinator merges into main, by plain merges. The raw-data repository is never force-pushed.
- *Traceability.* Notes and tables cite raw folders by commit. `delta_all.csv` names each row's source and state and is re-verified against them, with controls that must fail, and `check_numbers.py` ties each quoted number to a file.

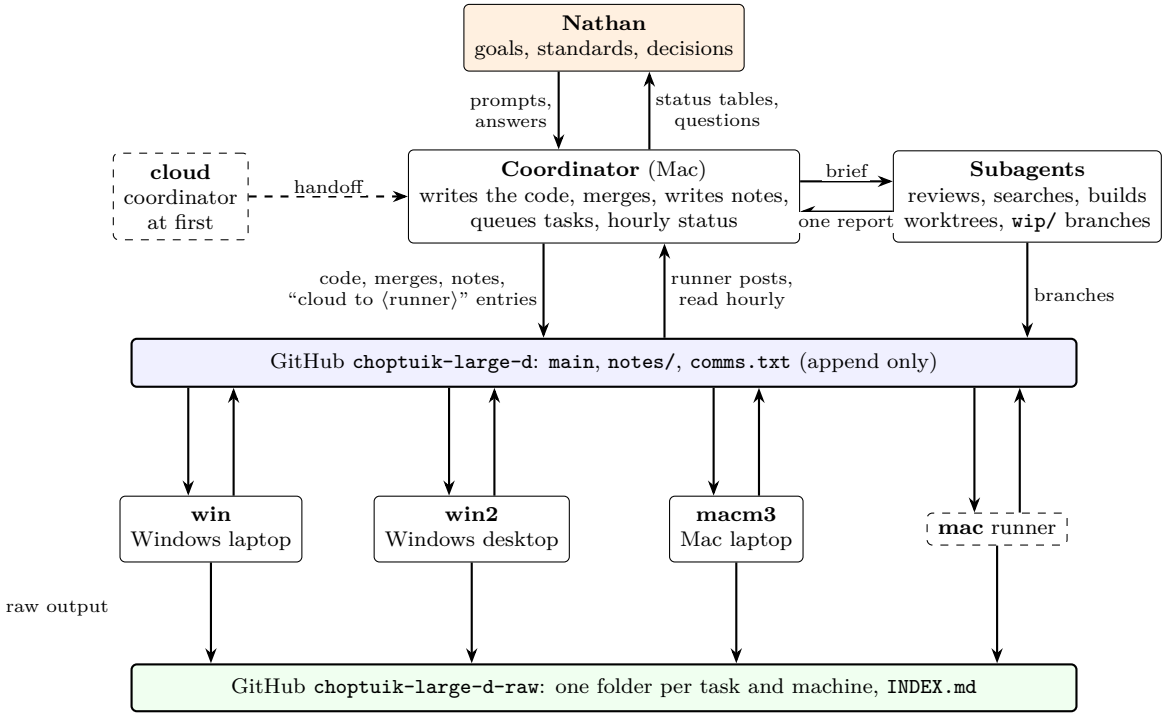


FIG. 1. The participants and how they were connected. I spoke only to the coordinator, and the coordinator spoke to the runners only through `comms.txt`. Each runner's watcher wakes it when an entry addressed to it arrives (down arrows), its posts carry its processor and memory use (up arrows), and its raw output goes to the raw-data repository. Dashed boxes are sessions that ended.

A claim can be followed back to the run that produced it, and any machine can rebuild the tables from clones with one command, `update_delta.py`.

- *Restarts and sleep.* A session that dies or a laptop that sleeps loses nothing that was pushed. Runners resume from the last pushed state, and the coordinator carried on across 29 context summaries and one change of machine from the log and the repositories.
- *Authorship and priority.* I keep the work in public repositories under my account, so every result, decision and message has a public, timestamped commit. Anyone can check the timeline, and it protects the work against being presented later as anyone else's, the provider of the model that did the computing included. Recent reporting describes researchers' fears that ideas typed into chatbots could feed future systems [12], and NIH treats putting an application's content into online AI tools as a breach of confidentiality [13]. Public also means that nothing private goes in.
- *The costs* are a pull before every write, merge conflicts on `INDEX.md` when two runners push at once, and a watcher that treats entries already present in a fresh clone as handled, so a new runner needs one bootstrap prompt.

Protocol.

- *Watchers.* A runner waits with `comms_watch.py`, which fetches every 60 s and exits when an entry addressed to it arrives, waking the session at no cost while it waits.
- *Status.* A recurring prompt asked the coordinator for a status every hour, which I shaped into one table row per runner and task, with the problem it serves and an ETA in hours.

Tasks. Each task entry states the following, here with examples from the entry that sent the octo-double chain to the desktop.

- *Commands*, exact and from a stated folder, such as `anm_run.py … --h_max 1.0 --D_max 200 --jet_o 1`.
- *Threads and priority*, such as "below normal, 18 threads".
- *Stop conditions*, such as "Stop at D_max, on a degenerate series, when two steps in a row need a of 0.125 or less, or after 10 h."
- *What to post*, such as the series time, the accepted step, the re-seed, the corrector's steps, max $|F|$ and Δ after the first two steps, then hourly, with processor and memory use.
- *What to push*, such as "Push T129-win2/ at each post."
- *Rules to apply without waiting*, such as "if the log gains no line for 20 minutes while one core is busy, take one

py-spy dump, kill the tree, restart from the last state with h_max halved (0.25, then 0.125), and post. Do not wait for me."

Long runs start with a pilot of the hardest case. A new value counts once another run, usually on another machine, reaches the same D, and on the large-D chain 18 such runs give the error column of 66 rows. A runner that sees a mismatch reports it and does not fix it.

IV. RULES OF EVIDENCE

I required that every check have a control that can fail, that discrepancies be reported rather than quietly fixed, and that thresholds never be loosened to pass, and the cloud session's handoff recorded these as my standing rules. AGENTS.md writes them out, partly following the verification protocols of BootLoops [11].

- Every check must be able to fail. Each is broken once on purpose, and the broken version stays runnable (Table II).
- Planted truth comes before real data.
- Every error bar says what it is.
- Independent checks share none of the machinery under test.
- Discrepancies are reported, not quietly fixed, and thresholds are not loosened to pass.
- Every quoted number is tied to a file by `check_numbers.py`.

TABLE II. Checks and their controls. Each control was run and failed as it must.

check	pass	control, which must fail	source
re-seed of a stored prediction (`checkq`)	converges, 2.25×10^{-37} at 153.43	the previous rule stops at 3.60×10^{-23}, exit 1	QUAD_DOUBLE 13.26–27
bit identity of a new build	28 of 28 jet outputs equal	a mutated constant, 28 of 28 differ	OCTO_DOUBLE 3
octo-double corrector from a planted start	6.86×10^{-42}, Δ to 1.18×10^{-34}	one term through the quad-double LU, stops at 3.39×10^{-12}	OCTO_DOUBLE 5.4
every value of Δ against its source	2445 rows equal	one row moved by one ulp, exit 1	QUAD_DOUBLE 13.25
low-D planner probe, smoke test	every case plans, exit 0	a cap of 0.001 s caps all, exit 1	LOW_D 16

Runners built the solver, checked each build bit for bit against stored references, ran the chains and the low-D legs, and flagged what did not match. They caught commands that did not run as written, expectations of the coordinator that were wrong, and a hang rule shorter than one step of their own runs. They also failed in ways the setup had to absorb. Machines slept on battery, and Claude Code stopped a watcher under memory pressure and would not restart it until I asked in that machine's own session, since an instruction of mine relayed through the log did not count.

Subagents. I asked for subagents where a task needed independence, breadth or hours beside the main work. My standing instruction is not to use them without asking and naming their number, and 26 of the 29 launches the coordinator's transcript records for this project followed a prompt of mine asking for one. They were six reviews and audits, six searches, six analyses, ten builds and this paper. Later briefs each named a git worktree and a backup branch, and the coordinator checked each report before relaying it.

V. HOW THE STUCK WORK MOVED

A. The stuck points

The work stuck at 31 points (Appendix A), from a toy simulation's period that came out 4% low to a residual floor of quad-double arithmetic near $D = 194$. They were of five sorts. The physics resisted a formula. The solver met walls in D that belonged to its formulation or its arithmetic (Fig. 2). The low-D runs grew slow and froze. The proofs and their certificate code needed independent review. And the setup itself stalled, when data were lost, machines slept or a runner sat idle. Of the 31, twenty were moved first by a prompt of mine, eight by the coordinator's own diagnosis and three by another session or an outside protocol.

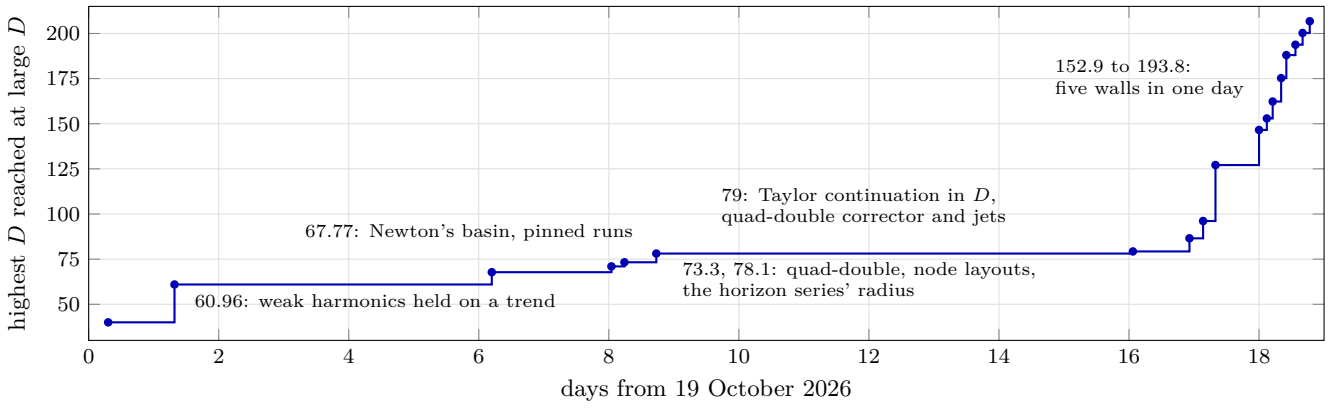


FIG. 2. The highest D reached at large D, from the runners' posts and the notes, with the walls named by what passed them. The flat stretch from day 9 to day 16 is the search for a formulation past 78 (Appendix A, rows 17 and 19 to 21).

TABLE III. Kinds of push among 145 prompts of mine that changed the course of the work, with verbatim examples. The full list, with each prompt's kind, is in `SOURCES.md`.

kind	n	examples	what it typically did
standards and process rules	25	"*from now on all agents should store the raw output in git somewhere different.*"; "*make sure to add precision tests for the quad-double solver.*"	a rule in AGENTS.md or the status format
resources	17	"*there is a new runner named win2, give it tasks*"; "*i can spool up another mac M3 laptop if you think it will help make the total work faster.*"	machines, runners, subagents, capacity
reframing	16	"*Could it be a sum?*"; "*we can map where these walls are and skip them*"	a form or route to test, sometimes rejected
asking why	15	"*then how to get to even 100? is there something wrong with our solver?*"; "*Why is large D calc on wormhole so much faster?*"	a diagnosis or a measurement
my own tools and projects	15	"*Does it make sense to rewrite in Rust?*"; "*could the residual qd stuff go inside multidouble?*"	the Rust kernel, `multidouble` releases
correcting Claude	15	"*the win machine is not doing anything?*"; "*i thought the purpose of the qd solver was to get in the larger D +70 range.*"	an idle machine, a drift or a wrong number caught
persistence	13	"*Yes continue to find the formula you are doing great don't give up. It exists*"; "*we need to keep going above 150*"	a stalled thread kept alive
raising the scale	12	"*just jump to 100*"; "*we can push past D=200 and go as large as we can*"	a limit removed, a decisive experiment
asking to understand	11	"*explain it as if a software developer wanted to know*"; "*What does collocation mean, what is the theory?*"	explanations, figures, an algorithm document
acting now	6	"*just make the quad-double solver, stop delaying.*"; "*have a subagent build the real thing and not a toy.*"	deliberation or a model ended

B. Kinds of push

Of all my prompts in the chat and in the four project sessions on the coordinator machine, 145 changed what was done next. Their wording sorts them into ten kinds (Table III), which overlap, so each prompt carries the kind it shows most.

Persistence and reframing carried the search for a formula. The prompts that kept it going did not say how. The reframings did, and Claude turned each into a test. Some held, as the sums of the chat did. Others did not. When I suggested that convergence was getting harder "*because maybe there is a repeating sum*", Claude tested whether the failed steps sat at whole values of the horizon exponent and found that they did not. When I asked whether the number Δ was approaching meant something, it listed constants within the error bar, and the later data excluded them.

Asking why produced the measurements behind the walls. After "*so i mean it is failing at those weak points, because the accuracy gets worse, so there must be a mathematical reason why*", the coordinator traced both the failures and the loss of accuracy to the weakest singular value of the Jacobian, which falls as D rises, and built a stall detector that I had asked for. Its runs showed $u\kappa$, the unit roundoff times the condition number, crossing 1 just before the stall at 60.96. After "*then how to get to even 100? is there something wrong with our solver?*" and "*I'm still not certain why we can't get past 78 D*", the walls became properties of the formulation, not of the solution, and that sent the search to other formulations and then to the literature.

Raising the scale turned questions into experiments. "*just jump to 100*" was answered by jumps of every size from two starts, and none landed past $D = 79.3$, which located the wall in Newton's basin rather than in the step rule. The same kind of prompt removed limits the work had set itself, such as a stop at $D = 200$.

Acting now ended deliberation. "*just make the quad-double solver, stop delaying.*" produced a quad-double dual, LU and solver with precision tests the same morning. "*have a subagent build the real thing and not a toy.*" moved the re-seeding of the horizon data from a model into the solver, where it later carried the chain past 150. One prompt acted ahead of need, "*yes add octo-double jets in hp engine, we don't need to run it until it is needed.*", and the octo-double series was ready when the quad-double one lost its digits.

My own tools and projects shaped the software. The kernel went to Rust with SIMD on my suggestion, the arithmetic moved into `multidouble`, the library I maintain and release, and figures followed the style of my other projects. Ideas also crossed between projects, as when I asked why the large-D computation of a related wormhole problem ran so much faster.

Resources, standards and corrections kept the setup running. I added machines and subagents when work queued, and asked that "*none of the agents should ever be not running tasks*". Standards came from failures. The raw-data rule followed a loss of records, and the status format followed tables that did not say what each task was for. Corrections caught what the coordinator missed, such as a laptop silent for 56 hours, a computer that had fallen asleep while the coordinator read its load as oversubscription, and the quad-double solver drifting from its purpose.

Asking to understand left documents behind. Prompts such as "*explain it as if a software developer wanted to know*" produced animated figures of one continuation step and a plain account of the algorithm in the README, which later moved into its own file.

C. What Claude brought

Within the direction I set, the coordinator's own diagnoses moved eight stuck points, and several others needed one after my push. Its methods recur.

- It replayed a failure on its stored input instead of waiting for a new run, which found two defects behind the stall at 152.93 within 34 minutes of my prompt.
- It tested a hypothesis before acting on it, as when the quad-double and octo-double Jacobians agreed to 1.2×10^{-36} and ruled out precision at 162.31.
- It built the next precision ahead of need when I asked, piloted it beside the running chain and switched with one entry.
- It asked for a stack dump before a kill, which put a hang somewhere other than where it had guessed.
- At a floor it could not pass, it measured the floor, accepted stalls below an explicit level with a flag and a second path, recorded that this weakens the convergence test, and had an octo-double corrector built to take over (`notes/OCTO_DOUBLE.md` 5–6).

D. How the pushes changed the method

Most of the rules in AGENTS.md trace to a stuck point.

- Pinning weak harmonics came from the stall at 60.96.
- The raw-data repository came from lost records.
- Bit-identity checks came from moving the arithmetic into a released library.
- Planted truth and mutation controls came from an outside protocol that exposed gaps.
- Second implementations came from proof reviews.
- Saved predictions came from the stall at 152.93, and hang rules from a hang.

My prompts set the direction and the pace, and the rules made the results checkable at that pace.

VI. THE RECORDS

TABLE IV. The records, at commit 07eff89 of the code repository and d0f3311 of the raw-data repository.

quantity	value
chat on claude.ai	62 prompts, a few on side projects
period of the repository	19 October 2026 to 6 November 2026; 19 days
commits on main	2227, of which 954 are entries of `comms.txt`
entries in `comms.txt`	974: coordinator 293, Windows laptop 318, Mac 221, Windows desktop 134, macm3 8
tasks; raw-data repository	T1 to T135; 124 folders, 548 commits
values of Δ in `hp/data/delta_all.csv`	2445 rows, from $D = 3.04375$ to 203.27
coordinator session on the Mac	334 prompts, 308 hourly status prompts, 29 context summaries, 30 subagent launches

VII. LIMITATIONS AND LESSONS

No expert has reviewed the work. I have not checked the mathematics myself. The checks of Sec. IV make the work testable, and they do not replace a reviewer.

The records are incomplete. The cloud sessions of the first week left no transcript on the coordinator machine, and the runners' transcripts stayed on their machines, so pushes of mine in those sessions are known only where the log relays them. The per-point records of 757 early points are lost.

The classification is mine. The kinds overlap, and another reader would draw the lines elsewhere. The counts say which pushes were common, not which mattered most.

The error bars are of several kinds. Differences between runs measure the path dependence of the held harmonics, about 10^{-20}, and the stall rule, 10^{-18} to 10^{-17}. Neither includes the bias of the held harmonics, 10^{-8} to 10^{-7} at $D = 79$ to 86, which the README states separately.

Lessons.

- Push on the goal, and let the method be argued.
- Ask why at every wall, and turn the answer into a measurement.
- Prefer a cheap decisive experiment to a long plan.
- Build the next tool before it is needed.
- Write the stop rule and the pass criteria before the run, and keep the control runnable.
- Save every input a failure depends on, and give every new value a second path on another machine.
- Keep everything in the repositories, where it is public, timestamped and recoverable.

VIII. HOW THIS WORK WAS DONE

I am the author of this work and of this paper. Claude, Anthropic's model, used through Claude Code, was the tool. It produced the code, the computations and the notes, as a coordinating session, runner sessions and subagents under my direction. This paper was drafted by a Claude subagent from the chat where the project began, the files and the transcripts named in `paper/ai/SOURCES.md`, and revised at my request after I read the earlier drafts. Its numbers come from files and from commands run on them. The references were checked against publisher, catalogue or repository records, and the one that could be checked only against a citing record is marked. No human expert has reviewed it. Corrections are welcome.

Data and code. https://github.com/nmoinvaz/choptuik-large-d and https://github.com/nmoinvaz/choptuik-large-d-raw.

Appendix A: The stuck points in order

In the column "moved by", a quotation is a prompt of mine, "relayed" marks one known through the log, and the other entries name the coordinator's diagnosis or another session. Sources for every row are in `SOURCES.md`.

	when	what stuck	moved by	resolution	what changed
1	18 Oct	toy simulation's Δ 4% low, one echo resolved	"*Explain the collapse numbers*"	literature check; the solution computed directly, 9 digits at $D = 4$	reproduce published values first
2	18 Oct	no formula from the first fits	"*…don't give up. It exists*"; "*Could it be a sum?*"	suggested forms fitted and tested on unseen D	held-out tests
3	18 Oct	scan stalls near $D = 9.3$	"*Continue searching*", then Claude's diagnosis	selection strength falls exponentially with D	no plain $1/D$ series; more precision
4	18 Oct	solver stalls near $D = 6$	Claude's diagnosis	weakly determined directions set aside	the weak directions become a theme
5	19 Oct	one minute per step on two cores	"*can we speed up the runtime by using SIMD?*"; a subagent	Rust kernel with SIMD, double-double residual	kernel in Rust, driver in Python
6	19 Oct	convergence harder with D	"*…maybe there is a repeating sum?*"	resonance tested and rejected	hypotheses tested openly
7	19 Oct	residual floor at $D = 28$	"*Do we need quad-double?*"	floor traced to one weak harmonic	solver and precision limits told apart
8	19 Oct	weak directions set Δ above $D = 23$; stalls at 27 to 31	Claude's diagnosis; a test on the laptop	weak-direction solve, damped steps, double-double	two machines, `comms.txt`, tasks
9	20 Oct	double-double climb stops at 60.96	Claude's diagnosis	harmonics 9 to 13 held on their trend	pinning
10	20–21 Oct	double solver fails below $D = 3.6$	Claude's diagnosis	double-double at high M, nodes moved	series tails recorded with each value
11	21 Oct	gaps in a certificate, an error bar and quoted numbers	an outside protocol (BootLoops)	mutation controls, planted truth, `check_numbers.py`	the rules of evidence
12	21 Oct	a certificate's claim wider than its proof	an independent review agent	claim narrowed to what the code proves	reviews by agents that did not write the code
13	25–26 Oct	chain stops at 67.77	my requests (relayed) and choices among options	derivative route measured and closed; pinned runs to 71	options posed with a recommendation
14	26 Oct	records of 757 points lost	"*is the data gone or not?*"; "*from now on all agents should store the raw output in git somewhere different.*"	search on both runners; raw-data repository	the raw-output rule
15	26–27 Oct	growing modes at $D = 4$ against the literature; counts at higher D on one layout each	a second layout; then "*what happened to our ghost stuff?*"	the modes were artifacts; recounts in a second layout	second-layout checks
16	27 Oct	pinned runs stall at 73.3	"*just make the quad-double solver, stop delaying.*"	quad-double solver with precision tests	arithmetic released in `multidouble` first
17	27 Oct	horizon block stops the chain past 73; layouts reach 78.1	"*continue to $D = 200$*"; "*try to skip past 73 by a larger number.*"	node layouts; Newton's basin measured	walls read as limits of the formulation
18	27 Oct–2 Nov	low-D Newton steps of 15044 s on the laptop	"*is it technically possible to go lower than 3.05?*"; a new desktop	continuation below 3.05, moved to win2	heavy runs to the always-on machine
19	2 Nov	wall at 78	"*just jump to 100*"; "*…is there something wrong with our solver?*"	jumps of every size fail past 79.3	cheap decisive experiments
20	3 Nov	collocation and a hybrid fail	"*I'm still not certain why we can't get past 78 D*"; a literature subagent	Taylor continuation in D [14]; 73.27 to 79.27	search the literature before building
21	4 Nov	double-double wall near 80	Claude's measurement; route subagents	quad-double corrector and jets	route subagents in parallel
22	3–4 Nov	low-D stall at 3.04375	"*…run a subagent, analyze our current method …*"	a stale checkpoint found; 3.04375 converged	analysis by a session that did not write the code
23	5 Nov	re-seeding only in a model	"*have a subagent build the real thing and not a toy.*"	re-seeding and an elimination corrector in the solver	models move into the code quickly
24	5 Nov	a theorem's certificate code without second implementations	"*create another subagent to do the next important thing you think is worthy*"	reimplementation from the statements alone	independent second implementations
25	5–6 Nov	low-D predictor polluted; planner frozen at $x = 0.0137$	Claude's diagnosis; "*can we queue it on a machine that is not busy?*"	lower order; probe with planted truth; a slow LU, not stuck	phase timers
26	5–6 Nov	restarts fail near 135	Claude's diagnosis	polish from all four parts of the state	full states stored and used
27	6 Nov	stall at 152.93	"*we need to keep going above 150*"	replay of the stored step; two defects fixed	predictions saved
28	6 Nov	stop at 162.31; re-seeds failing on win2	Claude's diagnosis; a runner's flag	hypothesis tested; new polish; looser divergence guard	comparison on stored inputs
29	6 Nov	quad-double series loses digits near 171	"*yes add octo-double jets in hp engine, we don't need to run it until it is needed.*" (earlier)	two-step pilot, switch to octo-double	build ahead, pilot, switch
30	6 Nov	hang above 188.02	a runner held it and dumped the stack	fail-fast; armed hang rule	dump before kill
31	6 Nov	quad-double floor at 193.77	"*we can push past D=200 and go as large as we can*"	stall rule with a flag; octo-double corrector	explicit, flagged acceptance

[1] M. W. Choptuik, "Universality and scaling in gravitational collapse of a massless scalar field", Phys. Rev. Lett. **70**, 9 (1993), doi:10.1103/PhysRevLett.70.9.

[2] N. Moinvaziri, `choptuik-large-d`, https://github.com/nmoinvaz/choptuik-large-d (2026).

[3] N. Moinvaziri, "Choptuik's echo period and critical exponent at large dimension: evidence for $\Delta_\infty = 1.643893746$ and an r^2-shift symmetry behind $\gamma_\infty = 1/2$", draft, `paper/physics/dss_physics.pdf` in Ref. [2].

[4] N. Moinvaziri, "The echo period of Choptuik's critical solution at large dimension: a selection rule and a computer-assisted proof", draft, `paper/math/dss_math.pdf` in Ref. [2].

[5] T. J. Dekker, "A floating-point technique for extending the available precision", Numer. Math. **18**, 224 (1971), doi:10.1007/BF01397083.

[6] Y. Hida, X. S. Li and D. H. Bailey, "Algorithms for quad-double precision floating point arithmetic", in *Proc. 15th IEEE Symposium on Computer Arithmetic (ARITH-15)*, 155–162 (2001).

[7] Anthropic, Claude Code documentation, https://code.claude.com/docs; on subagents, https://code.claude.com/docs/en/sub-agents (read 6 November 2026).

[8] N. Moinvaziri, the chat on claude.ai where the project began, 17–19 October 2026 (UTC), unpublished.

[9] N. Moinvaziri, `choptuik-large-d-raw`, https://github.com/nmoinvaz/choptuik-large-d-raw (2026).

[10] N. Moinvaziri, `multidouble`, https://github.com/nmoinvaz/multidouble.

[11] M. D. Schwartz, BootLoops, https://github.com/BootLoops-ai/bootloops (read at commit e16ba28 on 21 October 2026).

[12] C. Metz, "Could your chatbot be stealing ideas from you? It's a thorny question", The New York Times, reprinted in The Star, 11 September 2026, https://www.thestar.com.my/tech/tech-news/2026/09/11/could-your-chatbot-be-stealing-ideas-from-you-its-a-thorny-question (checked on the reprint).

[13] National Institutes of Health, "The Use of Generative Artificial Intelligence Technologies is Prohibited for the NIH Peer Review Process", notice NOT-OD-23-149 (2023), https://grants.nih.gov/grants/guide/notice-files/NOT-OD-23-149.html.

[14] B. Cochelin, "A path-following technique via an asymptotic-numerical method", Comput. Struct. **53**, 1181 (1994) (checked against a citing record only).